



Four Fixes on Base, No Migration

Zach Kelling

Lux Industries

2026

Four Fixes on Base, No Migration

Fifty lines of Solidity make Morpheus settlement real, with no chain move. It is the cheapest entry in this series.

The problem, in their numbers. MOR trades at \$1.95, market cap near \$17M [verified]. Real revenue is Lido and Aave yield, about \$476K a year and falling, down from \$3.24M a quarter at the 2024 launch [verified]. Emission outruns real revenue near ten to one [verified]. This is not a fraud: principal is withdrawable one to one. The honest problem is dilution outrunning revenue, over a settlement number no one can check. Settlement is `pricePerSecond` times elapsed time [verified]. The provider node signs a seven-field receipt, including the input and output token counts that say how much inference happened (`abi.go:12-19`) [verified]. The contract decodes five of the seven and drops the two that matter (`SessionRouter.sol:237`) [verified]. Only the provider signs (`SessionRouter.sol:227`). Nothing stops a provider opening a session against its own bid (`SessionRouter.sol:106`). The owner key, an anonymous five-of-nine, can rewrite any facet in one transaction, with no timelock [verified]. Morpheus built the honest half. The node already signs the work. The contract discards it.

The fix. Four edits to the `SessionRouter` facet in `Morpheus-Lumerin-Node/smart-contracts/`. None touch the token, the treasury, the staking math, or the chain.

- **Decode the whole receipt.** Add two `uint32` fields to the decode tuple at `SessionRouter.sol:237`. Settlement then reads the tokens, not just the clock. About five lines.
- **Make the consumer co-sign.** `closeSession` already holds `session.user` and the recover-and-compare helper (`SessionRouter.sol:543`). Run it against the consumer, and a provider cannot post volume the consumer will not sign.
- **Forbid self-dealing.** One `require(session.user != bid.provider)` removes the wash trade where a provider funds a session against its own bid.
- **Timelock the owner.** A timelock between the Safe and the owner role turns a stolen key into an announced upgrade with a withdrawal window, not an instant silent loss.

The outcome. About fifty lines of Solidity [illustrative: sum of the four edits above], on the diamond that already exists. No migration, no new token, no vote to move capital. Best case: settlement reads work the node already signs, both parties sign the bill, self-dealing is blocked, and a compromised owner key buys a window instead of the treasury.

Assumptions. Reading `inputTokens` proves a count was signed, not that the tokens were produced; attesting the work is a later step (LP-313). The receipt still settles on the provider's reported count until a consumer disputes it. A timelock narrows the owner key's blast radius; it does not remove it. This makes the base honest, not complete.

Relevance. Lux built `mord`, an un-fakeable settlement engine where faked work earns zero and model substitution is slashed; forty tests pass as of 2026-08-29 [verified]. We know which fifty lines, and why. This fix is the first rung, and the rest of the series stands on an honest receipt.

ADOPT-09. Facet `SessionRouter` in `Morpheus-Lumerin-Node/smart-contracts/`. Diamond `0x6aBE1d282f72B474E54527D93b979A4f64d3030a` on Base. Citations checked against the working tree. Leads to LP-313 and LP-323. • `research@lux.network`